이 매뉴얼로 시작하는 방법

목표와 완료 기준을 정하고, 작은 기능 하나를 구현·검증한 뒤, 확인된 결과를 기록한다. 이 흐름을 반복해 제품을 완성한다.

> **처음에는 01 → 10 순서로 읽고, 실제 작업에서는 남은 단계에서 시작하세요.**
> 모든 단계를 매번 별도 채팅으로 요청할 필요는 없습니다. 전체 작업을 맡기면 AI가 허용된 범위에서 이어가고, 사용자는 목표·중요한 선택·실제 결과를 확인합니다.

새 프로젝트
01부터 시작 → 일반 문서 구조 선택

기존 프로젝트
01에서 현재 코드·문서 확인 → 남은 단계

중단 후 재개
10의 재개 요청문 → 기록과 현재 상태 대조

AEOS 프로젝트
기존 .ai 계약 유지 → 일반 문서 중복 생성 금지

준비할 항목	사용자가 정할 내용
프로젝트	이름, 대상 사용자, 해결하려는 문제
작업 위치	새 폴더 또는 기존 저장소 경로. Work만 사용하는 기획 단계라면 원문 링크·파일
이번 도착점	설계만 / 구현·검증 / 배포 준비 / 실제 배포 중 어디까지인지
제약	기존 기술, 변경 금지 범위, 비용·데이터·외부 서비스 관련 조건

첫 실행 요청문 · Codex에 붙여넣기

```
$gon-project-lifecycle

[프로젝트 이름]을 시작하자.
목표: [누가 어떤 문제를 해결하는지]
대상: [실제 작업 폴더 또는 저장소]
이번 범위: [기획부터 구현·검증·배포 준비까지 등]
유지할 조건: [기술·데이터·변경 금지 범위]

현재 상태를 확인하고 이 프로젝트에 필요한 단계부터 진행해줘.
일반적인 가역 작업은 계속하고, 중요한 미정 사항만 확인해줘.
각 단계의 결과를 실제 파일·실행·검증 근거로 확인해줘.
웹 프로젝트라면 사용 가능한 gon-web-engineering 지침도 적용해줘.
기존 문서가 있으면 그대로 이어 쓰고 마지막에 다음 행동을 남겨줘.
```

대괄호 안을 실제 값으로 바꾸세요. “배포 준비”는 운영 환경 변경을 포함하지 않습니다. 이미 실제 배포를 요청하고 대상·범위가 명확하다면 같은 권한을 단계마다 다시 묻지 않습니다.

전체 흐름과 네 번의 확인

문서가 생겼다는 사실보다 다음 작업을 시작할 조건이 갖춰졌는지를 확인한다.

01 대상 확인 — 경로·상태·권한 › **02** 목표와 범위 — 사용자·MVP·성공 기준 › **03** 구조 설계 — 데이터·화면·오류 › **04** 작업 기반 — 문서·저장소·환경 › **05** 첫 작업 — 입출력·범위·검사

06 구현 — 작은 기능 완성 › **07** 실제 검증 — 실패 → 수정 → 재검사 › **08** 검토와 통합 — 리뷰·PR·CI › **09** 배포와 확인 — 허용 범위·운영 흐름 › **10** 기록과 재개 — 증거·다음 행동

위쪽 01–05를 마친 뒤 아래쪽 06–10으로 이어갑니다. 기능이 여러 개라면 05–08을 반복합니다. 07에서 실패하면 06으로 돌아갑니다.

상호작용형 전체 흐름도 열기 ↗

확인 지점	판단할 질문	충족하지 못했다면
G1 · 02 이후	첫 사용자가 얻을 결과와 제외 범위가 분명한가?	02에서 핵심 요구만 좁힌다.
G2 · 05 이후	어떤 파일을 바꾸고 무엇을 검사할지 실행 가능한가?	03–05에서 막힌 계약·환경만 정리한다.
G3 · 08 이후	해당 변경의 검사·필수 리뷰·통합 결과가 확인됐는가?	06–08로 돌아가 원인별로 수정한다.
G4 · 09 이후	실제 배포 대상과 관련 사용자 흐름이 확인됐는가?	준비/배포/운영 확인 중 현재 위치를 기록한다.

> 이 네 지점은 증거를 확인하는 기준입니다. 네 번의 별도 사용자 승인이나 자동화된 강제 차단 장치가 설치됐다는 뜻은 아닙니다.

규모에 맞게 줄이는 경로

요청	실제 진행
오탈자·단순 설정·짧은 문서 수정	현재 상태 확인 → 수정 → 결과 확인. 전체 문서 세트나 외부 리뷰를 추가하지 않는다.
재현 가능한 버그	01 → 05(재현 조건) → 06 → 07 → 08 → 10. 배포 요청이 있으면 09 포함.
다중 파일 새 기능·구조 변경	관련 설계·계획·테스트를 선택하고 완료 직전에 외부 리뷰 한 번.
조사·설계 문서만 요청	01–05 중 필요한 부분에서 종료. 제품 구현 완료로 표시하지 않는다.

도구와 이름을 구분하기

작업 방식, 사용하는 화면, 실행 장소, 모델을 각각 선택하면 인계와 권한 혼동이 줄어든다.

작업 방식

Chat · Work · Codex

판단 · 산출물 · 코드 작업

사용 화면

Desktop · 웹 · CLI · IDE

어느 인터페이스에서 작업할지

실행 장소

Local · Cloud

파일·도구·접속 권한이 있는 곳

모델

현재 선택된 모델과 추론 수준

작업 방식이나 스킬 이름과 별개

도구	이 매뉴얼에서 맡길 일	다음 단계에 전달할 것
사용자	제품 목표·범위·중요한 판단과 필요한 권한 결정	확정한 조건, 실제 결과에 대한 피드백
Chat	아이디어 정리, 대안 비교, 설명과 의사결정	결론·근거·미정 항목
Work	조사, 요구사항·설계·보고서 등 산출물 작성	접근 가능한 문서와 작업 명세
Codex / Codex CLI	저장소 확인, 코드 수정, 실행, 테스트, 디버깅	파일·diff·검사 결과·커밋/PR
Git / GitHub / CI	코드·문서 이력, 이슈·PR, 자동 검사와 릴리스 기록	대상 커밋과 해당 커밋의 실행 결과
Skills / MCP·Plugins	반복 절차 안내 / 실제 외부 도구와 자료 연결	읽은 지침과 도구 실행 결과

Chat·Work·Codex의 역할은 이 프로젝트의 운영 방식입니다. 기능이 완전히 분리된 것은 아니므로, 요구사항이 명확하면 Codex에서 기획·문서화까지 이어갈 수 있습니다. 매 단계마다 화면을 바꿀 필요는 없습니다.

이름	의미와 현재 기준
Gon AI Development OS	이 프로젝트에서 정리한 개발 진행 체계의 이름.
gon-project-lifecycle	실제 설치된 공통 절차 스킬. 시작·도입·개발·검증·전달·재개를 연결한다.
gon-web-engineering	실제 설치된 웹 기술 지침. 기존 스택을 보존하고 해당 작업에 필요한 지침만 적용한다.
ai-engineering-os	별도의 기존 보일러플레이트. PROJECT·ARCHITECT·.ai 계약을 사용하는 선택 경로.
gon-development-os	이 매뉴얼과 두 스킬, 빌드 도구·검증 이력을 함께 관리하는 현재 저장소. 초기 제안 이후 사용자 요청으로 저장소를 구성했다.

근거: 현재 설치된 두 SKILL.md 및 「3.Gon AI Development OS 표준 확인 경로 찾기」의 최종 정정. 스킬과 플러그인 역할·명시 호출은 OpenAI 공식 문서 참조.

10단계 중 01단계

01 작업 대상과 시작점을 확인한다

이번에 어디에서 무엇을 바꾸고 어디까지 진행할지 한 문장으로 정한다.

작업 위치 기획만 있으면 Work 또는 Chat · 코드가 있으면 그 폴더를 연 Codex

사용자가 준비할 것

프로젝트 이름·경로·기존 자료와 이번 도착점을 알려준다. 새 기획이라 경로가 아직 없으면 “기획부터, 저장소는 미정”이라고 적으면 된다.

AI가 수행할 순서

1. 신규·기존·재개 중 현재 상황을 구분한다. 설명만 요청했는지 실제 변경을 요청했는지도 확인한다.
2. 기존 프로젝트는 실제 저장소 루트, 브랜치, 미커밋 변경과 적용 중인 AGENTS.md·README를 읽는다. 기존 CLAUDE.md가 있다면 충돌 여부도 확인한다.
3. 사용 가능한 스킬·도구·파일 접근을 확인한다. Local과 Cloud, 현재 폴더와 worktree를 구분한다.
4. 이미 허용된 작업을 파악하고, 구현을 막는 불명확한 대상만 질문한다. 미커밋 변경을 보존한다.

받아야 할 결과

- 작업 대상 경로와 현재 상태의 짧은 요약
- 이번 완료 조건 한 문장
- 지금 시작할 단계와 중요한 미정 사항

다음 단계로 가는 기준

작업 대상과 변경 범위가 명확하고 기존 변경을 보존할 방법을 알면 02 또는 남은 단계로 간다.

그대로 사용할 요청문

```
$gon-project-lifecycle
이 프로젝트의 실제 경로, 현재 브랜치, 미커밋 변경,
기존 지침·문서·검사 명령과 사용 가능한 스킬을 확인해줘.
이번 목표는 [목표], 도착점은 [구현·검증/배포 준비/실제 배포]야.
완료 조건을 한 문장으로 정하고 필요한 단계부터 이어가줘.
```

예시

“시험지 OCR 서비스의 기존 업로드 화면에 재시도를 추가하고, 로컬 정상·실패 흐름 확인까지 진행”처럼 범위를 정한다.

막히면 폴더를 읽지 못하면 접근 가능한 원문·명세로 독립 작업을 진행하고, 실제 파일 작업에 필요한 경로 또는 연결만 요청한다.

02 목표와 첫 출시 범위를 정한다

누가 어떤 입력으로 어떤 결과를 얻는지를 먼저 정한다.

작업 위치 Chat·Work 또는 같은 Codex 채팅

사용자가 준비할 것

대상 사용자, 가장 불편한 문제, 제공할 입력·원하는 결과를 설명한다. “반드시 필요한 것”과 “나중에 해도 되는 것”을 구분한다.

AI가 수행할 순서

1. 사용자 문제를 한 문장으로 정리하고 대표 사용 흐름을 3–5개 행동으로 적는다.
2. 첫 출시의 최소 유효 범위(MVP)와 제외 범위를 나눈다. 미래 요구를 현재 기능에 섞지 않는다.
3. 수용 기준을 관찰 가능한 입력·행동·결과로 쓴다. 정확도·성능 같은 목표는 측정 대상과 방법까지 정한다.
4. 확정 결정·제안·미정을 구분한다. 구현에 당장 필요하지 않은 미정은 이유와 다음 확인 시점을 남긴다.

받아야 할 결과

- PRODUCT 또는 기존 요구사항 문서
- 기능별 수용 기준
- 비범위·미정 목록

다음 단계로 가는 기준

첫 사용 흐름과 성공·실패 기준을 구체적으로 설명할 수 있으면 G1 통과. 세부 미래 기능까지 확정할 필요는 없다.

그대로 사용할 요청문

```
[아이디어]를 첫 출시 가능한 범위로 정리해줘.
대상 사용자, 문제, 입력과 출력, 대표 사용 흐름,
이번 범위와 제외 범위, 관찰 가능한 수용 기준을 작성해줘.
확정·제안·미정을 구분하고, 개발을 막는 질문만 먼저 확인해줘.
```

예시

“사진 1장 업로드 → 인식한 텍스트 확인 → 사용자가 수정 → 저장”을 첫 범위로 잡는다. 해설 생성·결제·대량 처리는 별도 작업으로 미룰 수 있다. 이는 설명용 예시이며 실제 프로젝트 결정은 요구에 맞춰 정한다.

막히면 “좋게”, “빠르게”, “정확하게”만 있으면 대표 입력과 기대 결과를 추가한다. 데이터 없이 정확도 목표를 달성했다고 가정하지 않는다.

03 구조와 상태 흐름을 설계한다

작은 기능 하나가 화면·서버·데이터를 통과하는 경로와 실패했을 때의 동작을 정한다.

작업 위치 Work 또는 Codex · 웹 구현이면 gon-web-engineering 관련 지침

사용자가 준비할 것

유지해야 할 기술·외부 연동·운영 제약을 알려준다. 기존 프로젝트라면 현재 API·DB·인증 방식이 기준이다.

AI가 수행할 순서

1. 관련 코드와 기존 구조를 읽고 변경할 경계를 찾는다. 새 프로젝트의 기술은 실제 요구를 기준으로 선택한다.
2. 입력, 검증, 처리, 저장, 결과 반환을 연결하고 API·데이터 구조의 책임을 구분한다.
3. 빈 상태·대기·처리 중·성공·실패·재시도와 새로고침 후 복구를 필요한 만큼 설계한다.
4. 중요한 선택의 이유와 대안을 짧게 기록한다. 구조 결정이 바뀔 때만 결정 기록(ADR)을 남긴다.

받아야 할 결과

- ARCHITECTURE 또는 기존 구조 문서
- 화면·API·데이터의 계약과 상태 흐름
- 확정 기술과 남은 기술적 미정

다음 단계로 가는 기준

첫 기능의 입출력과 주요 오류 처리 방식이 정해지면 04로 간다. 전체 제품을 미리 세밀하게 설계할 필요는 없다.

그대로 사용할 요청문

```
확정한 MVP와 현재 저장소를 기준으로 첫 기능의 구조를 설계해줘.
화면·API·데이터 경계, 정상 흐름과 오류·재시도 흐름을 포함해줘.
기존 기술과 계약은 보존하고 변경이 필요한 이유를 기록해줘.
웹 작업이면 gon-web-engineering의 해당 지침을 적용해줘.
```

예시

신규 웹에서 기술이 미정이면 현재 웹 스킬은 React/TypeScript + Rust(Axum/Tokio) + PostgreSQL(SQLx)를 기본안으로 둔다. 기존 C#·Python 프로젝트를 이 조합으로 자동 변환하지 않는다.

막히면 외부 API나 인증 조건이 불명확하면 관련 계약을 먼저 확인한다. 조사만으로 접근 권한이나 실제 호출 성공을 확보했다고 기록하지 않는다.

04 문서와 실행 환경을 연결한다

다음 채팅에서도 같은 기준을 읽고 같은 검사를 실행할 수 있게 만든다.

작업 위치 실제 프로젝트 폴더를 연 Codex · 필요 시 GitHub 연결

사용자가 준비할 것

저장할 위치를 정한다. 새 원격 저장소 생성이 필요한 경우 이름과 공개 범위를 명확히 한다. 기존 프로젝트는 기존 문서·CI를 유지한다.

AI가 수행할 순서

1. 일반 프로젝트와 AEOS 구조 중 현재 상황에 맞는 한 가지를 선택한다. 같은 내용을 두 문서 체계에 중복 작성하지 않는다.
2. 기존 문서가 없고 필요할 때만 AGENTS·README·PRODUCT·ARCHITECTURE·PLAN·STATE를 구성한다. AI가 실제 내용과 명령을 채운다.
3. 매니페스트·잠금 파일·CI에서 실제 설치·실행·검사 방법을 확인하고 실행 가능한 최소 환경을 만든다.
4. 착수 전 검사 결과를 기록해 기존 실패와 새 회귀를 구분한다. 비밀값 대신 안전한 설정 방법만 문서화한다.

받아야 할 결과

- 문서 진입점과 실제 프로젝트 기준 문서
- 실행·검사 방법과 초기 결과
- 첫 작업의 선행 환경 조건

다음 단계로 가는 기준

다음 작업자가 문서 위치를 알고 필요한 명령을 실행할 수 있으면 05로 간다. 틀 생성만으로 내용 구성이 완료되지는 않는다.

그대로 사용할 요청문

```
이 프로젝트의 기존 문서와 개발 환경을 기준으로 작업 기반을 정리해줘.
같은 역할의 문서를 중복 생성하지 말고 필요한 내용만 보완해줘.
실제 설치·실행·검사 명령을 확인하고 기준선 결과를 기록해줘.
앱 구현까지 요청된 범위이므로 문서 정리 후 첫 기능으로 이어가줘.
```

예시

일반 프로젝트는 docs/PRODUCT.md 등에 기록한다. .ai/contracts·.ai/tasks·scripts/aeos.py가 보이면 AEOS 도입 상태를 확인하고 기존 체계를 이어 쓴다.

막히면 문서 생성기가 기존 파일을 감지하면 내용을 읽어 직접 연결한다. 경로 충돌·심볼릭 링크는 의도한 물리 경로를 확인한다. 설치되지 않은 명령을 실행 가능한 것으로 적지 않는다.

05 첫 작업을 검증 가능한 크기로 나눈다

한 번의 변경으로 관찰할 수 있는 결과를 정하고 구현 범위를 고정한다.

작업 위치 Codex · 기존 이슈/PLAN 또는 AEOS TASK·CONTEXT

사용자가 준비할 것

가장 먼저 확인하고 싶은 사용자 행동을 고른다. AI가 제안한 순서에서 목표를 바꿀 필요가 있는지만 판단하면 된다.

AI가 수행할 순서

1. 첫 기능의 입력·출력·허용 변경·제외 범위를 명시한다.
2. 선행 작업과 완료 조건을 적는다. 검증 방법이 없는 조건은 관찰 가능한 기준으로 바꾼다.
3. 버그는 수정 전에 실패 조건을 재현하고 정상 동작 기준을 기록한다.
4. 독립 작업의 위임이 명시적으로 요청되거나 적용 지침이 요구하는 경우에만 소유 파일과 통합 담당자를 정한다. 기본은 현재 AI가 직접 수행한다.

받아야 할 결과

- 구체적인 첫 작업과 검사 방법
- 관련 파일·자료·기준 revision
- 작업 순서와 제외 범위

다음 단계로 가는 기준

무엇을 바꾸고 무엇을 관찰하면 끝인지 분명하면 G2 통과. 구현을 요청했다면 여기서 멈추지 않고 06으로 간다.

그대로 사용할 요청문

```
첫 작업을 [사용자 행동]으로 잡고 구현해줘.
입출력, 허용 변경, 제외 범위와 수용 기준을 먼저 짧게 정리해줘.
각 기준을 확인할 검사 방법을 연결하고 관련 범위만 수정해줘.
버그라면 수정 전 실패 조건부터 확인해줘.
```

예시

“업로드 화면만 만들기”보다 “허용된 사진을 업로드하면 저장된 결과를 조회하고, 잘못된 형식은 거부하기”가 연결 상태를 검증하기 쉽다.

막히면 한 작업이 여러 독립 사용자 목적을 담고 있으면 나눈다. 파일 수가 적다는 이유만으로 작은 작업이라고 판단하지 않는다.

06 작은 기능을 끝까지 구현한다

사용자 행동 하나가 실제 결과로 이어지도록 필요한 구성만 구현한다.

작업 위치 Codex Desktop 또는 Codex CLI · 같은 실제 폴더와 브랜치

사용자가 준비할 것

구현 중 새 요구를 추가할 때는 “이번 범위에 포함”인지 “후속 작업”인지 알려준다. 사소한 구현 선택은 AI가 기존 관례에 맞춰 진행한다.

AI가 수행할 순서

1. 현재 코드의 진입점과 관련 테스트를 읽고 가장 작은 변경으로 기능을 연결한다.
2. 프론트엔드·API·저장을 필요한 만큼 함께 구현한다. 임시 화면이나 모의 데이터와 실제 연동을 구분한다.
3. 기존 스타일·패키지 관리자·API 계약을 유지하고 무관한 정리나 리팩터링을 섞지 않는다.
4. 작업 중 의미 있는 검사를 실행하고 실패 원인을 좁힌다. 도착한 결과와 다음 확인을 상태에 반영한다.

받아야 할 결과

- 요청과 직접 관련된 코드 변경
- 실행 가능한 첫 사용자 흐름
- 모의 처리·미연결·남은 항목의 구분

다음 단계로 가는 기준

구현된 흐름이 검증 가능한 상태이면 07로 간다. 코드 작성이나 프로세스 시작만으로 기능 완료라고 하지 않는다.

그대로 사용할 요청문

```
정리한 첫 작업을 구현해줘.
관련 코드·규칙을 먼저 확인하고 기존 변경을 보존해줘.
사용자 행동이 API와 저장 결과까지 이어지게 만들고,
임시 처리나 아직 연결되지 않은 부분은 명확히 표시해줘.
구현 후 관련 검증으로 계속 진행해줘.
```

예시

업로드 버튼 → 서버의 파일 형식 검사 → 저장 → 결과 조회를 연결한다. 업로드 성공 문구만 표시하고 저장이 안 되는 상태는 완료가 아니다.

막히면 동일 원인이 반복되면 가설을 바꾸고 관찰 근거를 추가한다. 의존 서비스가 막혔으면 가능한 독립 작업을 진행하고 차단 조건을 남긴다. 지연·할당량 때문에 모델을 임의로 바꾸지 않는다.

07 실제 동작을 검증하고 수정한다

수용 기준마다 실제로 무엇을 확인했는지 증거를 남긴다.

작업 위치 Codex · 테스트 도구 · 필요한 브라우저·실제 연동 환경

사용자가 준비할 것

대표 입력·실패 사례·접근 가능한 시험 환경을 제공한다. 로그인된 사용자 흐름이 필요하면 안전한 인증 경로를 사용한다.

AI가 수행할 순서

1. 변경에 맞는 빌드·타입 검사·테스트를 실행한다. 명령은 프로젝트와 CI의 실제 구성을 따른다.
2. 정상·빈 입력·잘못된 입력·중복/재시도 등 요구와 관련된 경계를 검증한다.
3. UI 변경은 관련 실제 사용자 흐름과 필요한 화면 크기·오류 상태를 확인한다. 적절한 연결 도구를 우선하고 웹 조작은 설치된 브라우저 도구를 사용한다.
4. 실패 원인을 수정하고 영향을 받는 검사를 다시 실행한다. 테스트 삭제나 기준 완화로 실패를 숨기지 않는다.
5. 대상 revision·환경·명령·결과·증거 경로를 기록하고 PASS/FAIL/NOT_RUN/UNKNOWN을 구분한다.

받아야 할 결과

- 수용 기준별 확인 결과
- 테스트 로그·화면·출력 등 필요한 증거
- 기존 실패와 새 실패, 미검증 환경의 구분

다음 단계로 가는 기준

필수 수용 기준을 확인했으면 08로 간다. 미실행 검사가 있으면 그 이유와 영향을 남기고 완료 범위를 제한한다.

그대로 사용할 요청문

```
수용 기준별로 실제 검증을 실행해줘.
정상 흐름과 관련 오류·재시도 흐름을 확인하고,
FAIL은 원인을 수정한 뒤 해당 검사를 다시 실행해줘.
각 결과에 대상 revision, 환경, 명령, 관찰 결과와 증거를 남겨줘.
실행하지 못한 검사는 NOT_RUN, 판단 근거가 부족하면 UNKNOWN으로 구분해줘.
```

예시

사진 업로드 후 새로고침해도 결과를 조회할 수 있는지 확인한다. 인증 화면으로 리다이렉트된 것만 확인했다면 로그인 이후 기능과 운영 화면은 미검증이다.

막히면 종료 코드 0·HTTP 200·설정 저장을 사용자 흐름 성공으로 확대하지 않는다. 브라우저마다 로그인 상태가 같다고 가정하지 말고 실제로 확인한 세션을 적는다.

08 검토하고 변경을 통합한다

요구사항·변경 내용·검증 결과가 서로 맞는지 검토하고 요청된 전달 단계까지 진행한다.

작업 위치 Codex · 적용되는 외부 리뷰 도구 · GitHub PR·CI

사용자가 준비할 것

커밋·PR·통합이 이번 요청에 포함됐는지 명확히 한다. 이미 허용한 범위는 다시 승인하지 않아도 된다.

AI가 수행할 순서

1. 관련 diff를 수용 기준과 비교하고 범위 밖 변경·비밀값·누락된 검사 여부를 확인한다.
2. 다중 파일 새 기능·구조 변경 또는 재현 가능한 버그라면 완료/커밋 직전에 외부 리뷰를 한 번 수행한다. 현재 외부 리뷰 지침·중앙 설정을 읽고 실제 반환 모델을 확인한다.
3. 지적은 근거로 수용 또는 기각한다. 수정한 부분은 관련 검사를 재실행한다. 단순 문서·설정 변경에는 외부 호출을 붙이지 않는다.
4. 허용 범위에서 작업 관련 변경만 stage하고 staged diff를 확인해 커밋·PR을 만든다.
5. 필수 CI·리뷰를 해당 revision에서 확인하고 허용된 merge까지 진행한다. 최신 변경 이후의 결과인지 확인한다.

받아야 할 결과

- 변경 목적과 실제 검증을 담은 PR 또는 완료 보고
- 실제 리뷰 결과와 미해결 사항
- 요청 범위에 따른 커밋 SHA·PR·통합 상태

다음 단계로 가는 기준

필수 검사·리뷰와 요청된 통합이 확인되면 G3 통과. PR 생성만 했거나 CI가 진행 중이면 그 상태로 기록한다.

그대로 사용할 요청문

현재 변경을 요구사항과 대조하고 적용 지침에 필요한 완료 경계 리뷰를 수행해줘.
리뷰 지적을 근거로 처리하고 수정분을 검증해줘.
이번 작업에 해당하는 변경만 커밋·PR로 전달해줘.
필수 CI와 허용된 통합까지 확인하고 실제 SHA·PR·결과를 보고해줘.

예시

“업로드 재시도 추가 / 중복 요청 재현 및 회귀 검사 통과 / 배포 미실행”처럼 PR에 최종 변경을 설명한다. 요청한 모델과 다른 리뷰 응답은 동일 리뷰의 통과로 처리하지 않는다.

막히면 리뷰 호출 실패는 미완료다. 필수 검사를 실행할 수 없으면 결과와 필요한 조건을 남긴다. 강제 push나 보호 규칙 우회는 일반적인 통합 요청에 포함되지 않는다.

09 배포 준비와 실제 배포를 확인한다

배포 가능 상태, 실제 배포 완료, 사용자 기능 확인을 각각 구분한다.

작업 위치 Codex · 기존 CI/CD · 지정된 실행 환경

사용자가 준비할 것

대상 환경과 이번 요청이 배포 준비인지 실제 배포인지 알려준다. 새 비용·운영 데이터 변경 등 기존 권한 밖의 결정을 정한다.

AI가 수행할 순서

1. 설정·의존성·배포 방법·필요한 데이터 변경과 기존 롤백 방법을 구체화한다.
2. 실제 호스트·프로젝트·Docker context 등 실행 대상을 확인한다. 예전 주소나 기본 context만 믿지 않는다.
3. “배포 준비” 요청이면 검토 가능한 변경·실행 절차·롤백 조건까지 준비해 전달한다.
4. 실제 배포가 허용됐으면 기존 CI/CD 경로로 진행하고 완료 결과·대상 revision을 확인한다. 권한 밖의 마지막 행동만 구체적인 결과를 제시한 뒤 확인한다.
5. 배포 후 상태 점검과 관련 실제 사용자/API 흐름을 확인한다. 실패 시 승인 범위의 복구 또는 롤백 절차를 따른다.

받아야 할 결과

- 대상 환경·배포 revision·실행 결과
- 핵심 사용자/API 흐름의 확인 결과
- 롤백 방법·남은 제한·운영 확인 상태

다음 단계로 가는 기준

실제 환경에서 요청한 동작을 확인하면 G4 통과. 배포 준비만 요청했다면 그 결과를 완료로 전달하고 운영 확인은 NOT_RUN으로 남긴다.

그대로 사용할 요청문

```
[환경·서비스]에 이번 변경을 배포하고 실제 동작까지 확인해줘.
대상과 배포 revision, 기존 CI/CD·롤백 경로를 먼저 확인해줘.
이미 허용된 범위에서 진행하고 새로운 권한이 필요한 마지막 행동만 알려줘.
배포 완료와 실제 사용자/API 흐름의 확인 결과를 각각 보고해줘.
```

예시

CI가 성공해 이미지가 만들어진 상태, 서버에 그 이미지가 올라간 상태, 사용자가 업로드·조회에 성공한 상태는 서로 다른 증거다.

막히면 운영 인증이나 환경 접근이 없으면 “배포 상태 확인 / 사용자 흐름 미검증”처럼 정확히 제한한다. localhost의 성공으로 운영 성공을 대신하지 않는다.

10 결과를 기록하고 다음 작업을 잇는다

다음 채팅이 전체 대화를 다시 읽지 않고 현재 파일과 결과를 대조해 이어갈 수 있게 한다.

작업 위치 Codex · 기존 STATE/이슈 · 필요 시 Work에서 보고서 정리

사용자가 준비할 것

이번 결과가 실제 목적에 맞는지 확인하고 다음 우선순위를 알려준다. 이어서 진행할 때는 프로젝트 경로와 남은 목표를 전달한다.

AI가 수행할 순서

1. 실제 구현·검증·리뷰·통합·배포 상태를 나눠 기록한다.
2. branch/revision·미커밋 변경·증거 위치·남은 문제·실패한 접근과 다음 행동을 짧게 남긴다.
3. 다음 채팅에서 상태 기록과 현재 Git·코드·필요한 CI를 대조하고 남은 단계부터 재개한다.
4. 반복 개선은 구체적인 실패와 효과가 확인됐을 때 제안한다. 전역 스킬·설정·메모리를 임의로 수정하지 않는다.
5. 예약 점검은 사용자가 요청했을 때 안정된 절차를 대상으로 구성한다. 변화가 없거나 같은 차단이면 조용히 종료하고 의미 있는 변화만 알리도록 한다.

받아야 할 결과

- 짧은 현재 상태와 증거 링크
- 다음 실행 가능한 작업 하나
- 재개 요청문 또는 필요한 인수인계

다음 단계로 가는 기준

다음 사람이 작업 위치·남은 일·확인 방법을 알 수 있으면 이번 단위가 끝난다. 미완료 항목은 완료로 바꾸지 않고 이어간다.

그대로 사용할 요청문

```
$gon-project-lifecycle
이 프로젝트를 이어서 진행해줘.
기존 상태 문서와 마지막 결과를 읽고 현재 경로·브랜치·변경·검증 상태와 대조해줘.
남은 것이 구현, 실패 해결, 리뷰, 배포 중 무엇인지 정하고 그 단계부터 진행해줘.
이전 작업을 다시 초기화하지 말고, 완료하면 실제 결과와 다음 행동을 갱신해줘.
```

예시

“업로드·조회 PASS, 재시도 PASS, PR 통합 완료, 운영 배포 NOT_RUN, 다음은 OCR 연결”이면 다음 작업은 업로드 화면을 다시 만드는 일이 아니다.

막히면 기록과 코드가 다르면 현재 관찰된 사실을 기준으로 수정한다. 오래됐다는 이유만으로 무조건 전체 테스트를 반복하지 않고 변경·실패·근거가 사라진 부분을 확인한다.

Work와 Codex 사이에서 전달할 내용

중요한 결정을 접근 가능한 문서에 남기고, 받는 쪽이 실제로 읽었는지 확인한다.

Chat / Work
요구·설계·결정
→
저장소 문서 / 원문
경로·버전·확정 범위
→
Codex
구현·검증
→
결과와 다음 작업
증거·PR·상태

전달 항목	필요한 내용
목표	사용자 문제, 이번 결과, 제외 범위
작업 위치	저장소·브랜치·실제 디렉터리
확정 결정	제품 규칙·기술·API·데이터 제약
미정	답이 없으면 막히는 항목과 후속으로 미룰 항목
완료 기준	대표 입력·행동·기대 결과와 확인 방법
현재 상태	완료/실패/미실행, 관련 revision·증거·PR
첫 행동	받는 사람이 바로 수행할 다음 작업

Work에서 인계 자료 만들기

지금까지 확정한 내용을 Codex 구현용 인계 자료로 정리해줘.
목표, 확정 결정, 미정, 입출력·API 계약, 허용 변경과 제외 범위,
수용 기준·검증 방법, 현재 상태와 첫 작업을 포함해줘.
원문 출처와 저장 위치를 연결하고 제안을 확정 사실처럼 쓰지 말아줘.
저장소에 직접 쓸 수 없다면 전달할 파일과 필요한 연결을 알려줘.

Codex에서 받기

$gon-project-lifecycle
[인계 자료의 실제 경로 또는 링크]를 읽고 현재 저장소와 대조해줘.
이번 목표와 확정 결정·미정, 사용할 기준 문서를 짧게 확인한 뒤
첫 미완료 작업을 구현·검증하고 결과를 기존 상태 기록에 남겨줘.

> 같은 계정·같은 프로젝트 이름만으로 같은 파일·대화·로그인·실행 환경이 확보됐다고 가정하지 않습니다. 명시한 문서를 읽은 결과와 현재 경로·브랜치로 인계를 확인하세요.

Desktop과 CLI를 오갈 때

실제 경로와 브랜치가 같으면 같은 파일을 이어서 작업할 수 있습니다. worktree라면 경로가 달라집니다. 동시에 같은 파일을 수정하지 말고 기존 작업을 마친 뒤 이어가거나, 요청된 독립 작업만 분리합니다.

터미널에서 시작하는 기본 형태

```
cd /실제/프로젝트/경로
codex
```

위 두 줄은 셸 명령입니다. $gon-project-lifecycle 요청은 Codex 입력창에 넣습니다. 설치·인증이 필요하다면 해당 환경의 현재 안내를 따르고 인증 값을 대화나 문서에 넣지 않습니다.

문서와 완료 증거를 관리하는 방법

프로젝트 기준은 저장소 문서로, 현재 판단은 실행 증거로 유지한다.

일반 프로젝트	무엇을 담는가	언제 갱신하는가
AGENTS.md	필수 규칙·진입 문서·검증·리뷰 조건	규칙이나 작업 진입점이 바뀔 때
README.md	설치·실행·검사·운영 방법	실제 사용 방법이 바뀔 때
docs/PRODUCT.md	사용자·문제·범위·수용 기준	제품 결정이 바뀔 때
docs/ARCHITECTURE.md	구성·데이터·인터페이스·선택 이유	구조 결정이 바뀔 때
docs/PLAN.md	작업 순서·의존성·완료 조건	우선순위나 남은 작업이 바뀔 때
docs/STATE.md	실제 결과·현재 제한·다음 행동	의미 있는 결과가 생겼을 때

기존에 SPEC·STATUS·이슈 등 같은 목적의 기록이 있다면 그 경로를 유지합니다. 작은 작업에 위 파일을 모두 만들지 않습니다. 이슈·PR이 상세 원장인 프로젝트는 STATE에 핵심 상태와 링크만 적습니다.

구현	검증	통합	배포	운영 확인
동작·diff	대상·실행 결과	SHA·CI·merge	환경·revision	실제 사용자 흐름

상태	의미	예시
PASS	직접 실행·관찰해 기준 충족	해당 revision에서 정상 업로드·조회 확인
FAIL	실행했고 기준 미충족	중복 재시도에서 동일 작업이 두 번 생성
NOT_RUN	실행하지 않음. 이유·영향 기록	운영 인증 부재로 로그인 이후 흐름 미실행
UNKNOWN	현재 관찰만으로 판단 불가	배포 요청은 접수됐지만 완료 상태 미확인

STATE에 남길 최소 기록 틀

확인 시점 / 작업 경로 / 브랜치 / revision / 미커밋 변경:
이번 목표와 범위:
구현된 동작:
검증: 대상 · 환경 · 명령/행동 · 결과 · 증거
리뷰 / PR / CI / 통합:
배포 대상 · revision · 실제 흐름 확인:
남은 문제와 미정:
다음 행동과 담당:
현재 허용된 작업 범위:

비밀값과 원문 전체를 넣는 기록장이 아닙니다. 재개에 필요한 사실과 증거 위치를 적습니다. 미래에 실행할 검사 결과를 미리 PASS로 채우지 않습니다.

기존 AI Engineering OS를 사용하는 경우

이미 .ai 계약을 사용하거나 템플릿 도입을 선택했다면 같은 Core를 유지한다. 일반 문서 구조로 다시 초기화하지 않는다.

내용	일반 프로젝트	AEOS
제품 목적·범위	docs/PRODUCT.md 또는 기존 명세	PROJECT.md
행동·검증 규칙	AGENTS.md	AGENTS.md
구조·인터페이스	docs/ARCHITECTURE.md	ARCHITECT.md
작업 입력	PLAN/이슈와 관련 문서	TASK.md + CONTEXT.md
구현·검증 결과	STATE/이슈/PR와 증거	RESULT.md + EVAL.md
검토·수용 판단	실제 리뷰·PR 승인 기록	REVIEW.md + DECISION.md

첫 작업의 실제 순서

1. 자신의 프로젝트 저장소에서 README·PROJECT·AGENTS·ARCHITECT를 읽고 제품 목적에 맞춘다. 템플릿 원본에 제품 코드를 작성하지 않는다.
2. 도구를 쓸 경우 현재 scripts/aeos.py와 의존 파일, 도움말을 확인한다. 수동 문서 사용은 별도 registry 승격 없이 가능하다.
3. 사용하지 않은 작업 ID로 여섯 계약의 틀을 만든다. 처음에는 TASK와 CONTEXT만 채운다.
4. 현재 AI가 계약을 읽고 구현·검증한 결과를 RESULT와 EVAL에 기록한다.
5. REVIEW와 최종 DECISION은 실제 독립 검토자 또는 사람이 담당한다. 실제로 없는 승인을 꾸며 쓰지 않는다.
6. 요청된 커밋·PR·CI·통합·배포를 거쳐 다음 작업으로 간다.

AEOS 저장소 루트에서 사용할 명령 형식

```
python3 scripts/aeos.py --help
python3 scripts/aeos.py validate
python3 scripts/aeos.py new-task TASK-001

# 선택 사항: 작업 입력을 초안 문서로 묶기
python3 scripts/aeos.py packet TASK-001 --runtime standalone --allow-unverified
```

Python 3.11+와 현재 CLI 의존성이 필요합니다. TASK-001은 예시이며 기존 ID가 있으면 새 ID를 고르세요. 실제 프로젝트의 도움말이 이 예시보다 우선합니다.

성공 표시	실제로 의미하는 것
validate PASS	registry·이력·증거 구조가 일관됨. 제품 테스트나 모델 접속 검사가 아님.
new-task 성공	작업 문서 6개 생성. 내용 완성이나 구현 완료가 아님.
DRAFT ONLY packet	입력 문서를 표준 출력으로 묶음. AI 호출·현재 모델 변경·후보 모델 승격을 하지 않음.
ACCEPT	수용 판단 기록. 별도의 배포나 외부 메시지 전송 권한을 추가하지 않음.

Standalone과 Orca는 같은 계약을 사용할 수 있습니다. Orca 워커 실행은 별도 요청과 현재 런타임 가이드가 있을 때 연결합니다. 기본 시작을 위해 병렬 워커나 모델 qualification을 먼저 구축할 필요는 없습니다.

근거: gon-project-lifecycle/references/aeos.md와 기존 템플릿의 docs/01-getting-started.md. 주가 예측 예제는 선택 참고 자료이며 제품 기능이나 필수 도입 요소가 아닙니다.

프로젝트 유형별로 완료 기준을 바꾼다

진행 순서는 공유하되 “잘 동작한다”를 판단하는 증거는 제품 유형에 맞춘다.

유형	작게 시작할 기능	완료를 확인할 핵심
웹·앱	입력 → API → 저장 → 결과 확인한 흐름	요구 기능, UI·오류·빈 상태, 관련 API·DB 계약, 필요한 접근성·화면 크기·실제 흐름
CLI·라이브러리	입력 한 종류를 처리하고 결과·오류 반환	설치·실행 재현성, 종료 코드·출력 계약, 오류 입력, 호환성, 배포 패키지의 실제 사용
AI Skill	대표 요청 하나의 절차와 결과	명시 호출, 적합/부적합 상황 구분, 기존 자료 보존, 반복 결과, 로더 발견과 실제 적용의 구분
AI Agent	도구 하나로 목표 하나 수행	작업 성공, 잘못된 도구 호출·권한 초과 방지, 실패 복구, 필요한 비용·시간·추적
MCP·Plugin	도구 하나의 입력과 결과 계약	스키마, 인증·권한, 정상·실패 응답, 실제 서비스 연동과 민감정보 보호

시험지 OCR로 따라가는 예시

작업 순서	사용자에게 보이는 결과	검증할 것
A · 업로드와 조회	사진을 올리면 저장된 항목을 다시 연다.	정상 형식, 잘못된 형식, 저장·조회 일치
B · OCR 연결	실제 사진에서 인식한 글자를 확인한다.	실제 연동, 대표 자료의 품질, 판독 불가 처리
C · 수정과 재시도	인식 결과를 고치고 실패 작업을 다시 실행한다.	수정 보존, 중복 작업 방지, 실패 후 복구
D · 승인된 후속 기능	해설·내보내기 등 확정된 기능을 추가한다.	제품 목적에 맞는 품질과 오류·한계 표시

각 작업에서 05–08을 반복하고, 릴리스 범위가 준비되면 09로 갑니다. OCR 정확도나 AI 해설 품질은 “API 응답을 받았다”로 판정하지 말고 대표 자료와 합의한 기준으로 평가합니다.

모델·위임·자동화의 기본 운영

- 현재 명시된 모델과 추론 수준, 적용 정책을 보존합니다. 이전 문서의 모델 추천표만으로 현재 세션을 바꾸지 않습니다.
- model-router 같은 별도 스킬은 실제 설치·활성화·호출 정책을 확인한 후 요청된 범위에서 사용합니다. 스킬을 읽는 것이 모델 실행이나 교체의 증거는 아닙니다.
- 위임은 사용자 또는 적용 지침이 요청한 독립 작업에 한정합니다. 소유 파일과 통합 책임을 정하고 결과를 주 작업에서 확인합니다.
- 수동으로 안정화된 점검만 요청에 따라 예약합니다. 주기·대상·권한·알림 조건과 실제 실행 결과를 확인합니다.

반복 절차를 스킬로 정리한 뒤 안정된 작업을 예약하는 원칙은 OpenAI 공식 안내와 일치합니다. 이 매뉴얼은 예약 작업을 생성하지 않습니다.

문제가 생겼을 때의 복구 순서

성공으로 간주할 근거가 없는 지점에서 원인을 좁히고, 가능한 작업은 계속한다.

상황	먼저 확인	다음 행동
스킬이 보이지 않음	현재 세션 목록·실제 경로·활성 설정·로더 오류	이름만 추측해 적용하지 말고 설치·발견 상태를 확인. 다른 환경이면 별도 확인.
Work 자료를 Codex가 못 읽음	링크·원문·저장소 접근 가능 여부	확정 범위·계약·검증 기준을 파일 또는 접근 가능한 원문으로 전달.
다른 파일이 보임	실제 경로·브랜치·worktree·Local/Cloud	올바른 작업 대상을 확인. 대화 동기화를 파일 복사로 이해하지 않기.
문서 틀만 생성됨	내용의 실제 값과 명령·링크	구현 요청이면 내용 연결 후 제품 구현·검증까지 계속.
테스트 실패	착수 전 기준선과 이번 diff	기존 실패와 회귀를 구분하고 관련 원인을 수정. 동일 가설 반복 금지.
외부 리뷰 실패	호출 결과·반환 모델·지침	미완료로 기록하고 필요한 조건 해결. 자동 모델 대체로 통과 처리 금지.
CI 성공, 화면은 실패	해당 revision·인증·실제 환경	사용자 흐름을 재현. YAML·HTTP 응답만으로 완료 판정 금지.
AEOS 생성기가 중단	부분 .ai·aeos.py·계약·registry 의존성	일반 생성기를 강제로 적용하지 말고 기존 체계를 확인. 가능한 수동 문서 작업 진행.
배포 권한이 부족	현재 요청·대상·남은 권한	diff·PR·검증·배포 계획을 준비한 뒤 마지막 필요한 권한만 확인.

차단을 풀기 위한 요청문

현재 막힌 지점을 실제 증거로 정리해줘.
관찰한 사실, 실패한 가설, 현재 완료된 범위,
독립적으로 진행할 수 있는 일, 다음에 필요한 조건을 구분해줘.
같은 시도를 반복하거나 미완료를 통과로 표시하지 말아줘.

완료 보고를 읽는 다섯 가지 질문

- 요청한 사용자 행동이 실제로 구현됐는가?
- 어떤 revision과 환경에서 무엇을 검사했는가?
- 미실행·실패·불확실한 항목이 구분됐는가?
- 커밋·PR·통합·배포·운영 확인 중 어디까지 도달했는가?
- 다음 사람이 바로 할 행동이 한 가지 이상 명확한가?

앞으로 매번 사용하는 짧은 운영 절차

사용자는 목표와 중요한 선택을 맡고, AI는 현재 상태 확인부터 결과 기록까지 이어간다.

시작할 때

경로 + 목표 + 범위 + 완료 기준

새 프로젝트면 첫 실행 요청문. 기존 작업이면 재개 요청문을 쓴다.

진행 중

실제 결과를 보고 방향 조정

새 요구는 이번 범위 또는 후속 작업으로 명시한다. 같은 문제는 같은 채팅에서 이어간다.

마칠 때

동작 + 검증 + 상태 + 다음 행동

현재 구현·검증·배포 수준을 확인하고 결과를 남긴다.

대부분의 작업에 쓰는 기본 요청문

$gon-project-lifecycle
[프로젝트 경로]의 현재 상태를 확인하고
[이번 목표]를 [완료 조건]까지 진행해줘.
기존 지침과 변경을 보존하고 필요한 검사·완료 경계 리뷰를 수행해줘.
실제 결과와 남은 제한, 다음 행동을 기록해줘.

작업을 마치거나 다른 채팅으로 넘어갈 때

이번 작업을 여기까지 정리해줘.
실제 완료한 동작, 실행한 검증과 증거,
리뷰·PR·배포 상태, 남은 문제와 다음 작업을 기존 상태 문서에 남겨줘.
다음 채팅에서 바로 이어갈 짧은 요청문도 작성해줘.

첫 적용의 성공 기준

처음에는 작은 기능 하나로 확인하세요. 스킬을 실제로 읽고, 기존 문서를 보존하며, 구현 후 검증 결과를 남기고, 다음 채팅에서 그 결과를 이어갈 수 있으면 첫 사용 흐름이 검증된 것입니다. 그다음 적용 범위를 늘리면 됩니다.

이 매뉴얼을 바꾸는 시점
실제 작업에서 반복된 누락이나 실패가 확인됐을 때, 원인·바꿀 절차·확인 방법을 정리해 수정합니다. 새 도구가 생겼다는 이유만으로 모든 프로젝트의 규칙을 자동 교체하지 않습니다.

기준일 · 출처 · 변경 이력

적용 기준과 원본 자료

2026년 10월 31일 프로젝트 대화·산출물·설치된 스킬을 통합한 매뉴얼이다. 현재 원본은 gon-development-os 저장소에서 관리한다.

대화에서 정리된 내용과 반영 위치

프로젝트 대화	매뉴얼에 반영한 내용
1.ChatGPT Codex Work 비교 Guide	작업 방식·화면·실행 장소·모델 구분, 인계, 파일·세션 경계, 브라우저와 예약 작업
2.개발 워크플로우 정리	통합 개발 흐름, 저장소 중심 기록, 작은 기능 반복, 유형별 완료 기준
3.Gon AI Development OS 표준 확인 경로 찾기	개발 체계·보일러플레이트·제안된 패키지 이름의 구분, 실제 스킬 중심 사용
gon-project-lifecycle 스킬 생성	시작·재개·검증·전달 절차, 문서 생성기, AEOS 연결, 검증 범위
스킬 저장 위치가 다른 이유 찾기	현재 두 개인 스킬 경로와 과거 설치 경로의 구분

긴 「1.ChatGPT Codex Work 비교 Guide」 응답은 대화 조회 도구의 메시지 길이 제한으로 끝부분이 일부 생략됐습니다. 실행 절차는 현재 설치된 스킬 원문과 대조했습니다. 원본 대화의 모든 문장을 그대로 재수록한 문서는 아닙니다.

현재 실행 기준으로 읽은 원본

- gon-project-lifecycle/SKILL.md — 공통 진입점과 단계별 연결
- project-start.md · work-cycle.md · delivery.md · aeos.md — 상세 절차
- gon-web-engineering/SKILL.md — 웹 구현 기본안과 기존 스택 보존
- AEOS 처음 시작하기 · AEOS 설계 방향 기록 — 기존 계약 사용법과 역사적 결정
- 스킬 생성·이동 검증 기록 · 기존 배포용 ZIP

이전 안내에서 현재 기준으로 정리한 항목

이전 내용	이 매뉴얼의 기준
Gon 표준과 ai-engineering-os를 같은 것으로 설명	통합 개발 방식과 기존 보일러플레이트를 구분한다.
스킬을 아직 만들어야 한다는 초기 제안	gon-project-lifecycle·gon-web-engineering은 현재 세션에서 읽을 수 있는 설치 스킬이다.
gon-project-lifecycle의 과거 .codex/skills 경로	개인 설치 위치는 ~/.agents/skills/gon-project-lifecycle이고 저장소 원본은 skills/gon-project-lifecycle이다.
gon-development-os 및 추가 전문 스킬 묶음	gon-development-os는 현재 관리 저장소로 구성했다. 추가 전문 스킬과 플러그인 패키징은 여전히 확장안이다.
모든 작업의 동일 10단계·고정 모델 추천표	규모에 맞는 단계만 적용하고, 명시 모델·현재 정책을 유지한다.
Work·로컬·플러그인의 자동 전달을 기대	환경별 발견·접근·실제 읽기·실행을 확인한다.

검증 기록의 의미

기존 기록의 스킬 파일 14개 해시가 최초 매뉴얼 제작 시점의 설치 파일과 모두 일치함을 확인했습니다. 원래 파일과 검증 기록은 archive/2026-10-31에 보관합니다. 기존 기록은 테스트 20개, 형식·링크·로더·AEOS smoke 검사 통과와 외부 리뷰 반영을 담고 있습니다. 최초 매뉴얼 제작에서는 테스트를 재실행하지 않았습니다. 저장소 통합 시의 새 검사와 이후 결과는 evidence 및 GitHub Actions 기록으로 구분합니다. 자동 스킬 선택, 실제 제품 구현·운영 배포, 다른 OS·Orca 런타임의 발견은 별도 확인 대상입니다.

공식 참조: Skills & Plugins · 개인·프로젝트 스킬 경로 · 반복 작업과 예약. 계정별 지원·권한·메모리 설정은 작업 시 확인하며, 이 매뉴얼을 사용하기 위해 임의로 설정을 변경하지 않습니다.